Funções Personalizadas 143
Testes e Boas Práticas: Arquivos de Teste 175
Programação Orientada a Objetos em Python 193
Quando as Coisas Dão Errado 253
Programando uma Aplicação com AMQP e RPC 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Página do Site 377

12

QUANDO AS COISAS DÃO ERRADO

A coisa mais certa em desenvolvimento de software é que, em algum momento, as coisas darão errado. Seria ótimo se todos os programas se comportassem bem o tempo todo, porém, esse é um sonho impossível. Há simplesmente problemas demais para antecipar; seu usuário pode, acidentalmente, entrar com um valor para o qual sua aplicação não estava pronta (por exemplo, uma string em um campo numérico ou uma data impossível como 30 de fevereiro), por exemplo.

Sempre que uma condição desse tipo ocorre, sua aplicação deveria, idealmente, tentar corrigir sozinha o erro e, se não for possível, informar ao usuário e sair do código problemático sem travar. Como citei, é praticamente impossível antecipar todas as falhas que podem ocorrer.

Criar código supondo que nada dará errado é uma temeridade. O melhor que pode fazer é ser cuidadoso ao programar e proteger seu código contra os possíveis problemas.

Neste capítulo, você conhecerá o mecanismo de ***exceções***, usado para lidar com as situações inesperadas em suas aplicações. Falarei primeiro das fontes mais comuns de erros, depois, como lidar com eles, usando o mecanismo de controle de exceções e, finalmente, como criar suas próprias exceções.

Os "criminosos de sempre"

Alguns tipos de código são, por natureza, mais sensíveis a falhas e devem ser criados com um cuidado extra. Posso destacar:

- ***Conexões com bancos de dados***: Pode acontecer uma falha no seu servidor de banco de dados que deixe seu ***SGBD***[1] temporariamente indisponível. Logo, todo código que acessa bancos de dados deveria ser protegido contra falhas.
- ***Injeção de código malicioso***: Não suponha jamais que todos os seus usuários são confiáveis. Em alguns casos, seu sistema estará exposto a usuários mal-intencionados, que tentarão tirá-lo do ar ou, até mesmo, roubar informações.
- ***Time-out também é um problema***: Se seu servidor web ou seu servidor de banco de dados demorar muito a responder, a aplicação pode travar.

Você lembra quando, ao longo do livro, em alguns capítulos, mostrei erros que aconteceriam quando algum código era escrito de um modo não recomendável? Esses erros eram todos ***exceções*** — não usei o termo técnico correto nas ocasiões para não adicionar complexidade no momento errado.

Alguns cenários de erro

Antes de tratar propriamente de exceções, mostrarei alguns códigos que podem resultar em erro. Considere a nossa "velha conhecida" classe Produto, definida no Capítulo 11. Reproduzo na Listagem 12.1, a última versão dessa classe, que foi criada na Listagem 11.7:

```
"""Classe que representa um produto"""

class Produto:
```

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

143
175
193
253
263
279
309
337
371
377

Você lembra quando, ao longo do livro, em alguns capítulos, mostrei erros que aconteceriam quando algum código era escrito de um modo não recomendável? Esses erros eram todos *exceções* — não usei o termo técnico correto nas ocasiões para não adicionar complexidade no momento errado.

Alguns cenários de erro

Antes de tratar propriamente de exceções, mostrarei alguns códigos que podem resultar em erro. Considere a nossa "velha conhecida" classe Produto, definida no Capítulo 11. Reproduzo na Listagem 12.1, a última versão dessa classe, que foi criada na Listagem 11.7:

```
"""Classe que representa um produto"""

class Produto:

def __init__(self, codigo: int, descricao:str, preco:
float)->None:

self.__codigo = codigo

self.__descricao = descricao

self.__preco = preco

self.__quantidade_estoque = 0

def entrada_estoque(self, quantidade: float)->None:

self.__quantidade_estoque += quantidade

def saida_estoque(self, quantidade: float)->None:

self.__quantidade_estoque -= quantidade

def visualizar_quantidade_em_estoque(self)->None:

print(f"A quantidade em estoque é {self.__quantidade_
estoque}")

@property

def codigo(self)->int:

return self.__codigo

@property

def descricao(self)->str:

return self.__descricao

@descricao.setter

def descricao(self, descricao: str)->None:

self.__descricao = descricao

@property

def preco(self)->float:
```

Expandir tudo | Recolher tudo

143
175
193
253
263
279
309
337
371
377

```
    def descricao(self, descricao: str)->None:
        self.__descricao = descricao
    @property
    def preco(self)->float:
        return self.__preco
    @preco.setter
    def preco(self, preco: float)->None:
        if preco <= 0:
            print('Erro: preço deve ser um valor positivo.')
        else:
            self.__preco = preco
    @property
    def quantidade_estoque(self)->float:
        return self.__quantidade_estoque
```

LISTAGEM 12.1: Classe Produto usando propriedades (produto5.py)

Abra o shell e digite os comandos da Figura 12.1.

```
 RESTART: /home/francisco/Dropbox/Alta Books/PythonWeb/código/cap12/produto5.py
>>> from produto5 import Produto
>>> prod = Produto(1, 'Notebook proc. i5 9ª geração', 5100.00)
>>> prod.entrada_estoque(10)
>>> prod.saida_estoque(12)
>>> prod.quantidade_estoque
-2
>>>
```

FIGURA 12.1: Criando um produto e entrando dados espúrios

Perceba que foi possível cadastrar uma saída que deixou o estoque negativo, o que expõe a falta de robustez da classe Produto. Não deveria ser possível realizar essa operação. Algumas abordagens que poderiam ser adotadas nesse caso:

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

143
175
193
253
263
279
309
337
371
377

FIGURA 12.1: Criando um produto e entrando dados espúrios

Perceba que foi possível cadastrar uma saída que deixou o estoque negativo, o que expõe a falta de robustez da classe Produto. Não deveria ser possível realizar essa operação. Algumas abordagens que poderiam ser adotadas nesse caso:

1. Não realizar a operação e não informar nada ao usuário. Isso poderia ser feito alterando o método saida_estoque() para:

```
def saida_estoque(self, quantidade: float)->None:
if (quantidade < self.__quantidade_estoque):
self.__quantidade_estoque -= quantidade
```

LISTAGEM 12.2: Método saida_estoque() que não permite estoque negativo (produto6.py)

Essa escolha não informa ao usuário do resultado, então, ele fica sem saber o resultado efetivo da sua retirada do estoque, por isso não é uma opção viável. Se você quiser testar o que acontece, na pasta de códigos-fonte deste capítulo encontrará um arquivo produto6.py, que tem a versão de saida_estoque() da Listagem 12.2.

Abra o arquivo, execute-o no IDLE e, em seguida, teste os comandos da Figura 12.2 no shell.

```
>>> from produto6 import Produto
>>> prod = Produto(1, 'Notebook proc. i5 9ª geração', 5100.00)
>>> prod.quantidade_estoque
0
>>> prod.entrada_estoque(10)
>>> prod.quantidade_estoque
10
>>> prod.saida_estoque(12)
>>> prod.quantidade_estoque
10
>>>
```

FIGURA 12.2: Não informar ao usuário e não permitir a operação

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

143
175
193
253
263
279
309
337
371
377

```
>>>
```

FIGURA 12.2: Não informar ao usuário e não permitir a operação

Primeiro, criei um produto e conferi que seu estoque inicial era zero(0); em seguida, dei entrada em 10 unidades no estoque dele, ficando com o total de 10 em estoque. Por fim, tentei realizar uma saída de 12 unidades do estoque: como não havia estoque suficiente, a solicitação não foi realizada e o estoque permaneceu com 10 unidades. Perceba que nada foi informado sobre o não atendimento da requisição, o que é uma falha *grave* do programa — a operação não foi realizada e nada foi informado ao desenvolvedor usuário da classe.

2. Mostrar uma mensagem de erro:

Essa é, geralmente, a primeira ideia dos desenvolvedores novatos — e ela não está errada! Apenas, digamos, "incompleta". Vejo três problemas aqui:

- Nenhuma informação é retornada ao código chamador. A mensagem é útil ao usuário final do sistema, porém, qualquer outra classe que porventura utilize os serviços de Produto não será informada acerca da ocorrência do erro.
- Acoplamento forte entre a classe Produto e a interface com o usuário. Se, por exemplo, seu sistema fosse migrado para uma plataforma online, seria trabalhoso aproveitar a mensagem gerada para esse erro específico.
- Mistura de aspectos: há código de apresentação (a mensagem de erro) e código de negócios (o trecho que verifica se o estoque é suficiente para atender à requisição) — lembre-se do "princípio da responsabilidade única": sua classe deve fazer uma e não mais que uma coisa.

3. Retornar um valor especial ou código de erro:

Essa opção era muito usada nos sistemas escritos no final dos anos 1980 — e você verá por que foi abandonada. Suponha que declarasse saida_estoque() como na Listagem 12.3.

```
def saida_estoque(self, quantidade: float).->bool:
resultado = True
if (quantidade < self.__quantidade_estoque):
self.__quantidade_estoque .= quantidade
else:
resultado = False
return resultado
```

LISTAGEM 12.3: Método saida_estoque() que retorna booleano indicando erro (produto7.py)

Se, com o tempo, você começasse a adicionar novos códigos de erro (por exemplo, em vez de um booleano, um inteiro, que conteria: 0- operação com sucesso; 1- estoque insuficiente; 2- produto inexistente etc.), a manutenção do sistema viraria um pesadelo.

"Houston, we've got a problem...": levantando exceções

A resposta a esses problemas, adotada não só por Python, mas por várias outras linguagens de programação, está nas chamadas "*exceções*". Grosso modo, pode-se pensar em uma exceção como uma parada na execução do código, efetuada pelo interpretador quando uma condição anormal ocorre.

Uma exceção é, como tudo mais em Python, um *objeto*. Todas as exceções herdam de uma classe comum (*BaseException*), o que permite tratá-las de forma homogênea em seus códigos.

Para criar suas próprias exceções (diz-se "levantar" ou "lançar" a exceção tal), primeiro você deve criar uma classe que herda de Exception, classe ancestral de todas as exceções em Python. Por exemplo, poderíamos criar a

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

Funções Personalizadas 143
Testes e Boas Práticas: Arquivos de Teste 175
Programação Orientada a Objetos em Python 193
Gestão de Casos Não Errados 253
Programando uma Aplicação com AMQP e RabbitMQ 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Página do Site 377

A resposta a esses problemas, adotada não só por Python, mas por várias outras linguagens de programação, está nas chamadas "*exceções*". Grosso modo, pode-se pensar em uma exceção como uma parada na execução do código, efetuada pelo interpretador quando uma condição anormal ocorre.

Uma exceção é, como tudo mais em Python, um ***objeto***. Todas as exceções herdam de uma classe comum (*BaseException*), o que permite tratá-las de forma homogênea em seus códigos.

Para criar suas próprias exceções (diz-se "levantar" ou "lançar" a exceção tal), primeiro você deve criar uma classe que herda de Exception, classe ancestral de todas as exceções em Python. Por exemplo, poderíamos criar a exceção EstoqueInsuficienteException para o caso de alguém tentar dar uma saída no estoque de um produto com uma quantidade maior que a disponível. Veja o código da Listagem 12.4:

```
"""Exceção lançada quando se tenta realizar uma saída do estoque para a qual
não há quantidade suficiente do produto disponível """
class EstoqueInsuficienteException(BaseException):
def __init__(self, mensagem):
self.mensagem = mensagem
def __str__(self):
return self.mensagem
```

LISTAGEM 12.4: Exceção personalizada *EstoqueInsuficienteException* (excecoes.py)

Em seguida, altere seu código para importar a classe da nova exceção criada e utilize uma instrução ***raise*** para levantar a exceção. No caso atual, você só precisa alterar o método saida_estoque(), mostrado na Listagem 12.3, para que comece com:

```
from excecoes import EstoqueInsuficienteException
```

e altere o corpo do método saida_estoque() para o exibido na Listagem 12.5.

```
def saida_estoque(self, quantidade: float)->None:
if (quantidade < self.__quantidade_estoque):
self.__quantidade_estoque -= quantidade
else:
raise EstoqueInsuficienteException("Não há estoque suficiente para atender a essa saída.")
```

LISTAGEM 12.5: Método saida_estoque() que lança uma exceção personalizada (produto8.py)

Para testar a nova versão de estoque.py, abra o shell e digite:

```
from produto8 import Produto
prod = Produto(1, 'Notebook proc. i5 9ª geração', 5100.00)
prod.entrada_estoque(10)
prod.saida_estoque(12)
```

Observe que, agora, a exceção é lançada no lugar do código de erro, como mostra a Figura 12.3.

```
>>> from produto8 import Produto
```

Python e Django
Francisco Marcelo de...

Expandir tudo | Recolher tudo

143
175
193
253
263
279
309
337
371
377

```
from produto8 import Produto
prod = Produto(1, 'Notebook proc. i5 9ª geração', 5100.00)
prod.entrada_estoque(10)
prod.saida_estoque(12)
```

Observe que, agora, a exceção é lançada no lugar do código de erro, como mostra a Figura 12.3.

```
>>> from produto8 import Produto
>>> prod = Produto(1, 'Notebook proc. i5 9ª geração', 5100.00)
>>> prod.entrada_estoque(10)
>>> prod.saida_estoque(12)
Traceback (most recent call last):
  File "<pyshell#162>", line 1, in <module>
    prod.saida_estoque(12)
  File "/home/francisco/[redacted]/código/cap12/produto8.py", line
20, in saida_estoque
    raise EstoqueInsuficienteException('Nao ha estoque suficiente para atender a essa
saida.')
excecoes.EstoqueInsuficienteException: Nao ha estoque suficiente para atender a essa
saida.
>>>
```

FIGURA 12.3. Testando a classe Estoque com o lançamento de uma exceção personalizada

Built-in Errors

Python já possui várias exceções prontas, que servem a uma variedade de situações. Eles são conhecidos como ***built-in errors***. Por exemplo, ao tentar abrir um arquivo com a função `open()` em modo de leitura("r"), se o arquivo não existir, será levantada uma exceção IOError.

Tratando exceções com try...except...else...finally

O mecanismo de tratamento de exceções da linguagem Python é o que garante que você lide adequadamente com os erros que porventura ocorrerem em seus programas e tome providências adequadas para recuperar, se possível, o funcionamento do código, como, por exemplo, exibindo uma mensagem para o usuário.

Funções Personalizadas 143
Leitura e Gravação Arquivos de Texto 175
Programação Orientada a Objetos em Python 193
Gestão de Casos Não Errados 253
Programando uma Aplicação com SMTP e XML 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Coordenando a Aplicação de Me 377

Tratando exceções com try...except...else...finally

O mecanismo de tratamento de exceções da linguagem Python é o que garante que você lide adequadamente com os erros que porventura ocorrerem em seus programas e tome providências adequadas para recuperar, se possível, o funcionamento do código, como, por exemplo, exibindo uma mensagem para o usuário.

A forma mais simples de tratamento de exceções é com um bloco try...except:

```
try:
# suite que pode gerar um erro aqui
except tipo_da_exceção:
# código que manipulará a exceção
```

Quando um suite é protegido por uma estrutura try...except, o código dele é executado e, se ocorrer um erro durante essa execução, o controle passa ao código após o except, em que a exceção pode ser capturada e tratada. A Listagem 12.6 provoca uma exceção ao tentar abrir um arquivo inexistente. Para tratar essa situação, protegi o código potencialmente falho — a parte que abre o arquivo — com um bloco try...except. Ao tentar executar esse código, ao invés da "horrível" mensagem padrão do Python, é exibida outra (bem) mais elegante:

```
Arquivo abcxyz.txt não encontrado.
"""Tratando exceções"""
try:
with open('abcxyz.txt', 'r') as arquivo:
for linha in arquivo:
print(linha, end='')
except IOError:
print('Arquivo abcxyz.txt não encontrado.')
```

LISTAGEM 12.6. Tratando exceções com try...except (excecoes2.py)

Você pode, também, encadear vários blocos except para tratar tipos diferentes de exceções, como na Listagem 12.7.

```
"""Tratamento de exceções"""
try:
n1 = int(input('Digite um número: '))
n2 = int(input('Digite outro número: '))
resultado = n1/n2
except ZeroDivisionError:
print('Não é possível dividir por zero.')
except ValueError:
print('Digite um inteiro válido.')
```

Funções Personalizadas 143
Laços e Iteração Arquivos de Texto 175
Programação Orientada a Objetos em Python 193
Gestão de Casos Não Errados 253
Programando uma Aplicação com AMQP e API 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Concluindo a Aplicação de 377

```
print('Não é possível dividir por zero.')
except ValueError:
print('Digite um inteiro válido.')
except:
print('Entrada inválida.')
else:
print(f'A entrada executou sem erros. A divisão de {n1} por {n2} resulta em {resultado}.')
finally:
print('O programa terminou!')
```

LISTAGEM 12.7: Tratando exceções com try...except...else...finally (excecoes3.py)

O programa da Listagem 12.7 é bastante rico em exemplos de tratamento de exceções:
Primeiro, protegi um suite incluindo-o emum bloco ***try***, o que garante que, se ocorrer algum erro no processamento das instruções no suite, ele será adequadamente tratado.
Em seguida, testei se alguns ***built-in errors*** ocorreram e, em caso afirmativo, mostrei uma mensagem ao usuário. Perceba que a mensagem varia de acordo com o erro capturado:

- ZeroDivisionError: Se o segundo número digitado for zero, a divisão não será possível e gerará uma exceção ZeroDivisionError. Nesse caso, o usuário verá a mensagem: "Não é possível dividir por zero."
- ValueError: Se for digitado um valor que não possa ser convertido em um inteiro (uma letra, por exemplo), será lançada essa exceção. Nesse caso, o programa exibirá: "Digite um inteiro válido."
- Se ocorrer qualquer outro erro não específico, ele será capturado no suite except. Esse suite costuma ser usado para o caso de surgirem exceções não previstas pelo programador. Lembre-se de ***nunca colocar as exceções mais específicas*** (ZeroDivisionError e ValueError, por exemplo) ***antes da exceção genérica*** (suite except), senão elas nunca serão tratadas.
- else: Se presente, o código desse suite será executado, se não acontecer nenhum erro no bloco protegido pelo try (alguns iniciantes se confundem com esse comportamento pois pensam que o else funcionaria como outro suite except, o que **NÃO** é o caso).

O suite finally

Quando há uma cláusula finally em um suite de tratamento de exceções, o código desta será executado, havendo erro ou não. Esse tipo de suite costuma ser usado para colocar código de liberação de recursos, como, por exemplo, fechar um arquivo aberto.

DICA

Você pode tratar mais de uma exceção no mesmo suite, reaproveitando o código do tratamento. Para tal, use uma tupla que agrupa os erros como neste exemplo:

```
try:
arquivo = open('abcxyz.txt','r')
for linha in arquivo:
print(linha)
except (IOError, TypeError) as erro:
```

Funções Personalizadas 143
Lendo e Gravando Arquivos de Texto 175
Programação Orientada a Objetos em Python 193
Gestão de Casos Não Errados 253
Programando uma Aplicação com AMQP e JAVA 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Construindo a Página de Abertura 377

```
for linha in arquivo:
print(linha)
except (IOError, TypeError) as erro:
print("Erro ocorrido: ", erro)
finally:
arquivo.close()
```

Repassando exceções

Às vezes é útil capturar uma exceção, realizar algum tratamento em um suite except e levantar novamente a mesma exceção para que seja tratada pelo programa chamador do código atual. Para isso, simplesmente use uma instrução raise sem argumentos (a mesma instrução que usei para lançar uma exceção personalizada na classe Produto, na Listagem 12.5). A Listagem 12.8 mostra um exemplo:

```
"""Tratamento de exceções - exemplo de propagação de erro com raise """
try:
with open('abcxyz.txt', 'r') as arquivo:
for linha in arquivo:
print(linha, end='')
except ZeroDivisionError:
print('Não e possivel dividir por zero.')
except ValueError:
print('Digite um inteiro válido.')
except:
print('Erro inesperado.')
raise
else:
print('O programa executou sem erros.')
```

LISTAGEM 12.8: Repassando uma exceção com raise (exemplo_raise.py)

Exercícios resolvidos

1. Que tipo de exceção será levantada se você tentar acessar um arquivo inexistente com a função open()? O que se pode fazer para evitar o problema?
   Solução: O arquivo a ser aberto pode não existir. Isso pode ser evitado protegendo a chamada em um bloco try...except.
2. Ao acessar arquivos com open(), é importante fechá-los após o uso para que os dados neles contidos não sejam corrompidos. Como garantir que os arquivos serão fechados ao final do uso?

Solução: Pode-se proteger o bloco que manipula o arquivo em um try...finally e colocar o fechamento do arquivo no suite do finally, o que garante que ele será executado.

Expandir tudo | Recolher tudo

Funções Personalizadas 143
Lendo e Gravando Arquivos de Texto 175
Programação Orientada a Objetos em Python 193
Gestão de Casos Não Errados 253
Programando uma Aplicação com AMQP, E-Mail 263
O Django Framework 279
Modelos 309
Formulários e Templates 337
Criando Telas de Cadastro com o Django Admin 371
Coordenando a Aplicação de Um 377

```
print(linha, end='')
except ZeroDivisionError:
print('Não é possível dividir por zero.')
except ValueError:
print('Digite um inteiro válido.')
except:
print('Erro inesperado.')
raise
else:
print('O programa executou sem erros.')
```

LISTAGEM 12.8: Repassando uma exceção com raise (exemplo_raise.py)

Exercícios resolvidos

1. Que tipo de exceção será levantada se você tentar acessar um arquivo inexistente com a função open()? O que se pode fazer para evitar o problema?
   Solução: O arquivo a ser aberto pode não existir. Isso pode ser evitado protegendo a chamada em um bloco try...except.
2. Ao acessar arquivos com open(), é importante fechá-los após o uso para que os dados neles contidos não sejam corrompidos. Como garantir que os arquivos serão fechados ao final do uso?

Solução: Pode-se proteger o bloco que manipula o arquivo em um try...finally e colocar o fechamento do arquivo no suite do finally, o que garante que ele será executado.

Exercícios propostos

1. Considere o seguinte código:

```
try:
arquivo = open('abcxyz.txt','r')
for linha in arquivo:
print(linha)
except IOError:
print('Arquivo abcxyz.txt não encontrado.')
```

   Altere-o para que, após a mensagem personalizada, imprima a exceção levantada pelo Python quando o arquivo não foi localizado.
2. Altere novamente o código da questão 1, desta vez para que a exceção gerada seja repassada ao programa que chamou o código.

- *SGBD* é a sigla de *Sistema Gerenciador de Banco de Dados*, a ferramenta que controla as informações da sua aplicação, se elas estiverem armazenadas em um banco de dados.